

India News Intelligence Platform

Comprehensive System Documentation

1. What We Have Built

We have built a **Production-Grade, Multi-Agent AI News Intelligence Platform**. It is designed to operate like a professional, autonomous digital newsroom. The system runs 24/7 with minimal human intervention, collecting the most important news stories from across India, verifying their facts, deduplicating them, ranking them by importance, and publishing them as beautiful summaries to a Discord server and a web dashboard.

Key Deliverables:

- **15 Specialized AI Agents:** Each agent is responsible for a single, distinct task (e.g., deduplication, fact-verification, ranking, monitoring).
 - **28 Integrated News Sources:** Pulling from major Indian publications (The Hindu, NDTV, Economic Times, Livemint, etc.).
 - **Discord Bot:** Offers 13 interactive slash commands (like `/news` , `/finance` , `/search`) and auto-publishes top stories as rich embeds.
 - **Real-Time Web Dashboard:** A glassmorphism-themed UI showing agent health, system statistics, and recent publications in real-time.
 - **Robust Persistence:** An asynchronous SQLite database that stores articles, performance metrics, heartbeats, and incidents without data loss.
-

2. How It Was Built (Architecture & Tech Stack)

The platform is built on an **Event-Driven Architecture (EDA)**. Instead of agents directly calling each other (which causes tight coupling and cascading failures), they communicate by publishing and subscribing to an asynchronous **Event Bus**.

Tech Stack:

- **Core Language:** Python 3.10+ with `asyncio` for high-concurrency tasks.
 - **Database:** SQLite (via `aiosqlite`) with Write-Ahead Logging (WAL) enabled for high-performance concurrent reads/writes.
 - **Task Scheduling:** `APScheduler` triggers the news pipeline on a recurring schedule (default every 2 hours).
 - **LLM Integration:** Abstracted LLM clients supporting Google Gemini, OpenAI, Groq, and Ollama.
 - **Web Dashboard:** `aiohttp` for the backend API and static file serving, combined with Vanilla JS, HTML, and CSS (Canvas for charts).
 - **Discord Integration:** `discord.py` for bot interactions and rich embeds.
 - **Machine Learning:** `sentence-transformers` (specifically `all-MiniLM-L6-v2`) running locally for semantic similarity calculations (no API cost).
-

3. Step-by-Step Function of the System

Every 2 hours (or when triggered manually via the `/news` command), the system executes a meticulously orchestrated pipeline:

Step 3.1: Orchestration (`ops_manager.py`)

The **Operations Manager** creates a unique `run_id` and starts the pipeline. It ensures no two pipelines run at the exact same time.

Step 3.2: Source Discovery & Collection (`source_discovery.py` , `rss_worker.py`)

- The **Source Discovery** agent commands the **RSS Worker** to fetch news from 28 configured sources concurrently.
- If a source fails, the **Recovery Engine** applies retry logic. If the RSS fails entirely, it can optionally fall back to a headless browser (`browser_worker.py`).
- *Output*: A large pool of raw, unverified articles.

Step 3.3: Deduplication (`dedup_agent.py`)

- **Stage 1**: Exact URL matches are removed.
- **Stage 2**: Exact Title hashes are removed.
- **Stage 3: Semantic Clustering**. The agent uses a local ML model (`sentence-transformers`) to convert headlines into vectors. Articles with >82% cosine similarity are clustered as the "same story". It keeps only the article from the most credible source (e.g., PIB over a random blog).

Step 3.4: Fact Verification (`fact_verifier.py`)

- Detects clickbait patterns using regex (e.g., "You won't believe...").
- Calculates a **Corroboration Score**: It checks if multiple independent publications are reporting the same keywords. A story covered by 3 independent sources gets a massive credibility boost.
- *Output*: Articles below a 50% confidence score are discarded.

Step 3.5: AI Summarization (`summarizer.py`)

- The remaining high-quality articles are sent to the LLM (Gemini/Groq/OpenAI).
- The LLM acts as a professional news editor. It writes a non-sensational headline and a strictly factual, 2-3 sentence summary (max 80 words).
- If the LLM is offline or rate-limited, the system safely falls back to keyword-based category matching and the raw RSS descriptions.

Step 3.6: Algorithmic Ranking (`ranker.py`)

Articles are scored on 9 dimensions without using an LLM (to save time and cost):

1. **Category Weight**: (e.g., National Security > General News).
2. **Impact Keywords**: (e.g., "Earthquake", "Budget", "Elected").
3. **Freshness**: Newer articles score higher.
4. **Concrete Data**: Titles with numbers/percentages (e.g., "GDP grows 7%") score higher.
5. **Source Priority**: Tier 1 sources rank higher than Tier 3 sources.

Step 3.7: Quality Assurance (`qa_agent.py`)

The **QA Agent** acts as the final gatekeeper. It checks:

- Is the headline length appropriate?
- Is the URL valid?
- Did we accidentally post this exact headline in the last 48 hours?
- Anything failing QA is rejected.

Step 3.8: Publication (`discord_bot/publisher.py`)

The top 10 surviving stories are saved to the database and pushed to Discord as beautiful, color-coded embeds with credibility badges.

Continuous Background Processes

While the pipeline runs, other agents never sleep:

- **Health Monitor:** Pings the internet, Discord, and all 28 news sources every 5 minutes.
 - **Executive Supervisor:** Monitors the "heartbeat" of all 15 agents. If an agent freezes, the supervisor restarts it.
 - **Failure Analyzer:** Logs every error into a central `incidents` table for later review.
-

4. How to Operate the System

4.1 Web Dashboard (UI)

- Open your browser to `http://localhost:8080`.
- The dashboard updates automatically every 30 seconds.
- You can monitor:
 - **Agent Health:** See which agents are online, idle, or failed.
 - **Stats:** Total articles fetched vs. published.
 - **Incidents:** Any background errors that were caught and recovered from.
 - **Latest Stories:** What was published today.

4.2 Discord Commands

Interact with your bot in Discord by typing `/` to see available commands:

- `/news` — Force the pipeline to run right now.
- `/breaking` — Get the top 5 highest-priority stories instantly.
- `/finance`, `/technology`, `/politics`, `/sports`, etc. — Get the top 5 stories in a specific category.
- `/search <keyword>` — Search the database for specific news (e.g., `/search RBI`).
- `/summary` — View performance statistics of the AI newsroom.

4.3 Terminal Operations

- **Start the system:** `python main.py` (Runs the 24/7 newsroom and Discord bot).
 - **Test the system safely:** `python main.py --dry-run --once` (Runs one collection cycle and prints results to your terminal without sending anything to Discord. Great for testing).
-

5. Room for Improvement

While the platform is production-ready, AI systems can always be expanded. Here are the top recommendations for v2.0:

1. **RAG (Retrieval-Augmented Generation):** Currently, the summarizer only looks at the text provided by the RSS feed. If we allow the `browser_worker.py` to scrape the full article text and store it in a Vector Database (like Milvus or Pinecone), the `/search` command could answer complex questions (e.g., *"What were the key takeaways from the Union Budget across all articles?"*).
2. **Automated Newsletter Generation:** Add a `NewsletterAgent` that runs once at 8:00 AM every day, aggregating the top 10 stories into a cohesive, beautifully formatted email using SMTP or

SendGrid, rather than just Discord.

3. **Sentiment Analysis:** Implement a lightweight sentiment model (like `VADER` or a HuggingFace pipeline) to classify articles as Positive, Negative, or Neutral. The ranker could then be tuned to ensure the top 10 stories aren't overwhelmingly negative, providing a more balanced news diet.
4. **Multi-Agent Debating (Advanced Fact-Checking):** Instead of a single Fact Verifier, introduce two LLMs that debate the factual accuracy of controversial stories before publication. If they cannot reach a consensus, the story is flagged for human review.
5. **Cloud Deployment (Docker):** While `docker-compose.yml` is provided, deploying this to AWS ECS or Google Cloud Run with an external managed database (like PostgreSQL instead of SQLite) would allow the platform to scale to thousands of users horizontally.